Experiment 8: Perform DCL and TCL Commands

Aim: Understanding Data Control Language (GRANT, REVOKE, User Privileges) and Transaction Control Language (COMMIT, ROLLBACK, SAVEPOINT, Isolation Levels).

1 [DCL — User Creation & GRANT]: Write SQL statements to create two database users ('lab_analyst'@'localhost' and 'lab_guest'@'localhost') and grant SELECT privileges on ODIMatch and Player to 'lab_guest'.

```
-- Root connection: Create users and grant access
CREATE USER 'lab_analyst'@'localhost' IDENTIFIED BY 'AnalystPass123!';
CREATE USER 'lab_guest'@'localhost' IDENTIFIED BY 'GuestPass123!';
GRANT SELECT ON aarya61_match.ODIMatch TO 'lab_guest'@'localhost';
GRANT SELECT ON aarya61_match.Player TO 'lab_guest'@'localhost';
SHOW GRANTS FOR 'lab_guest'@'localhost';
```

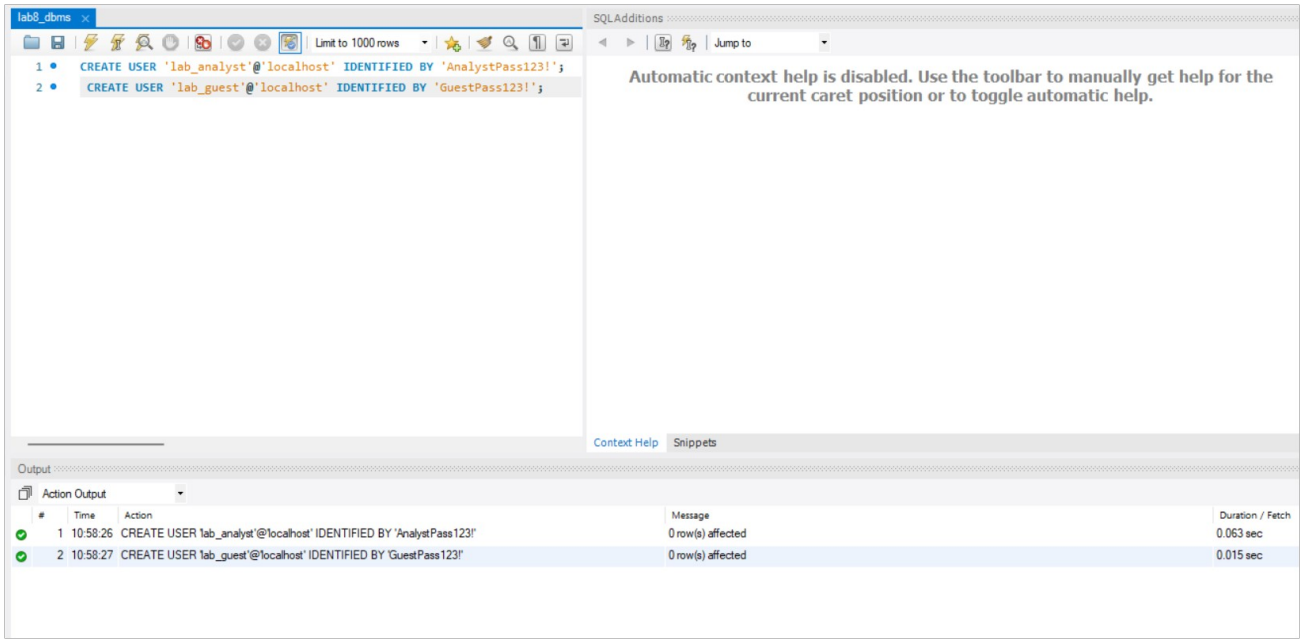


Fig. 1.1: Both MySQL user accounts created successfully.

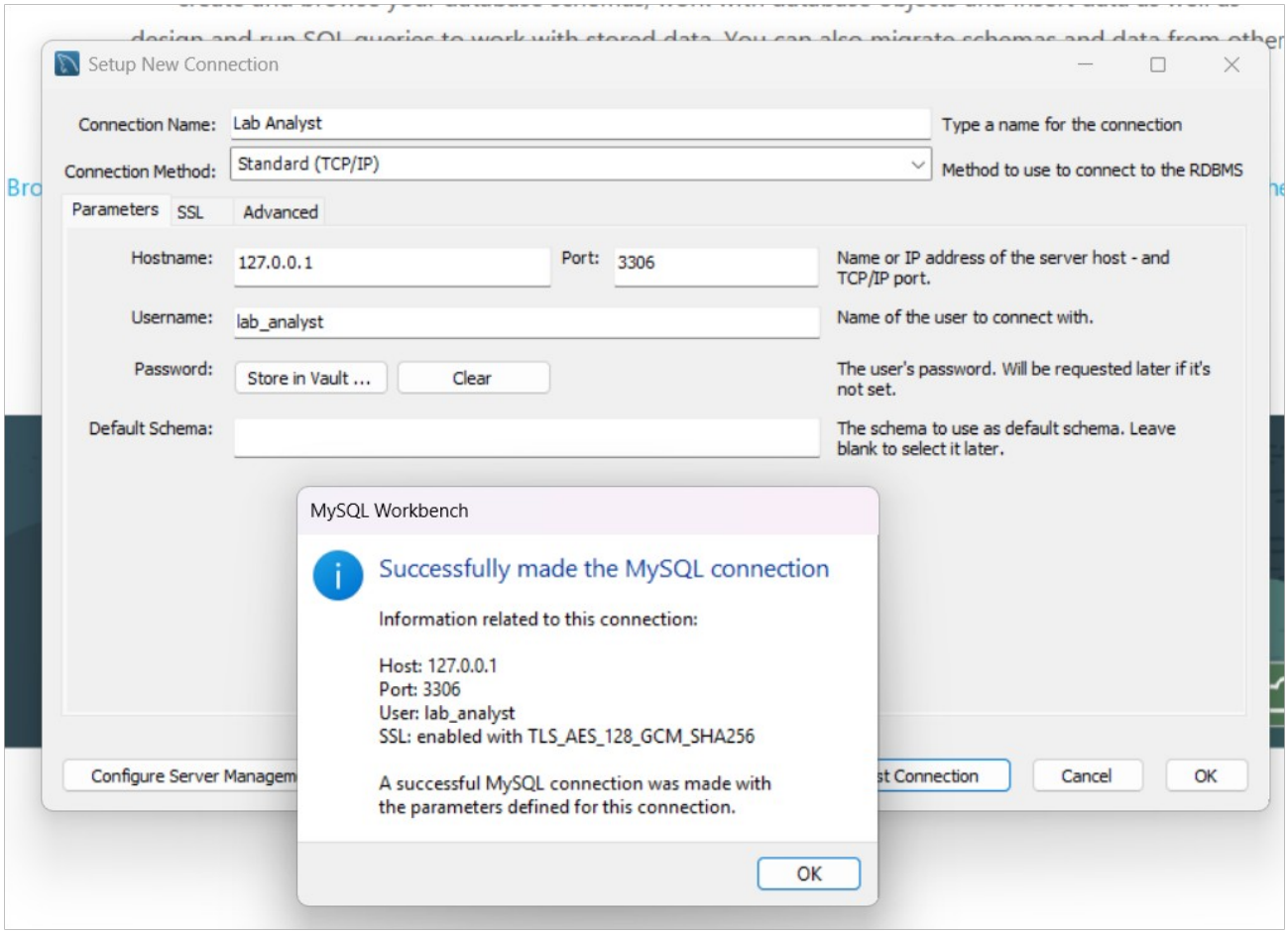


Fig. 1.2: Lab Analyst connection tested successfully.

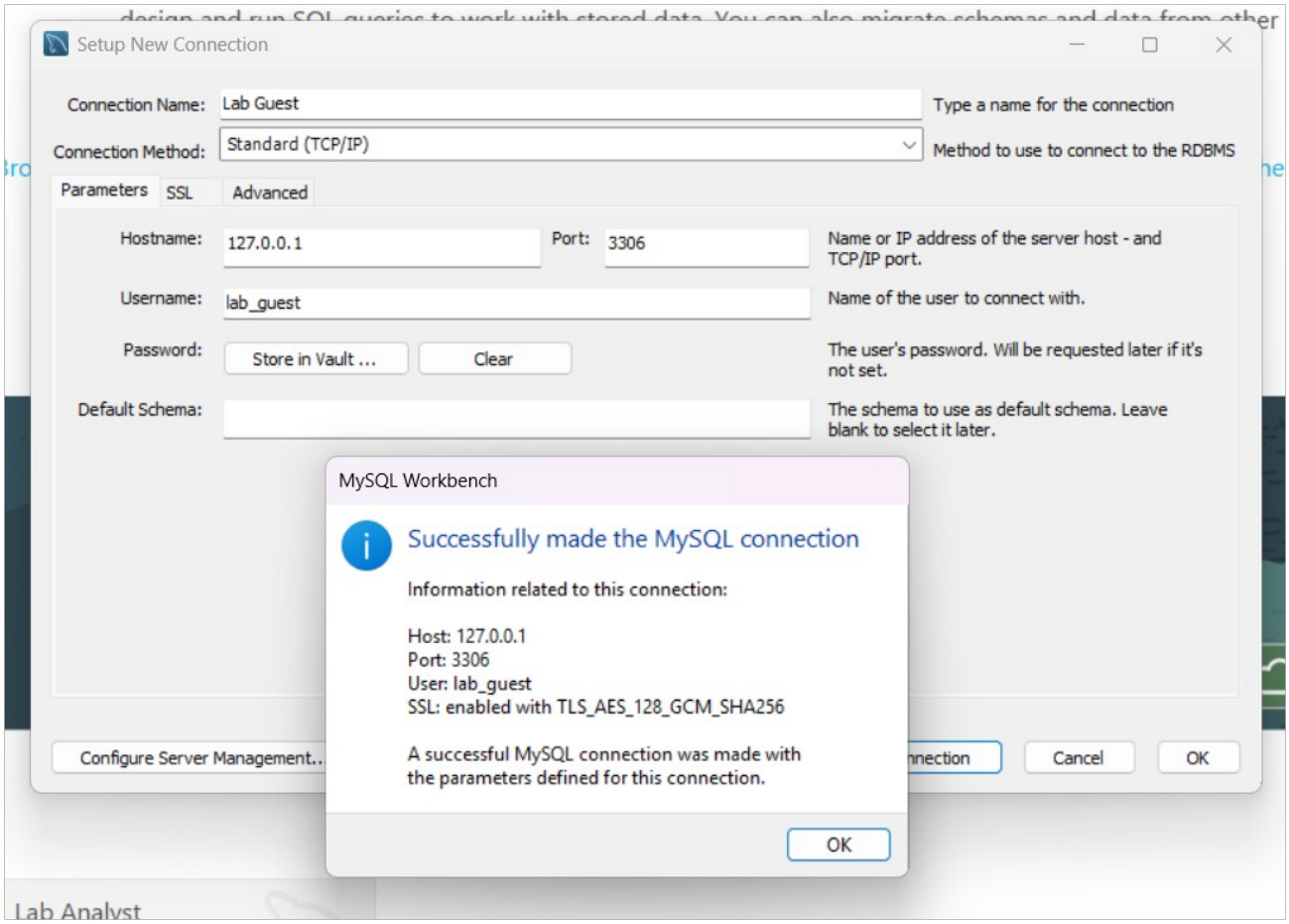


Fig. 1.3: Lab Guest connection tested successfully.

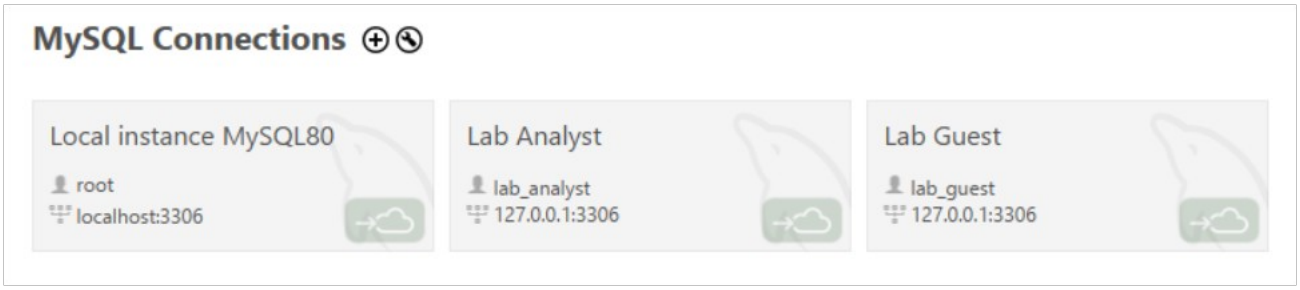


Fig. 1.4: Root, Lab Analyst and Lab Guest connections.

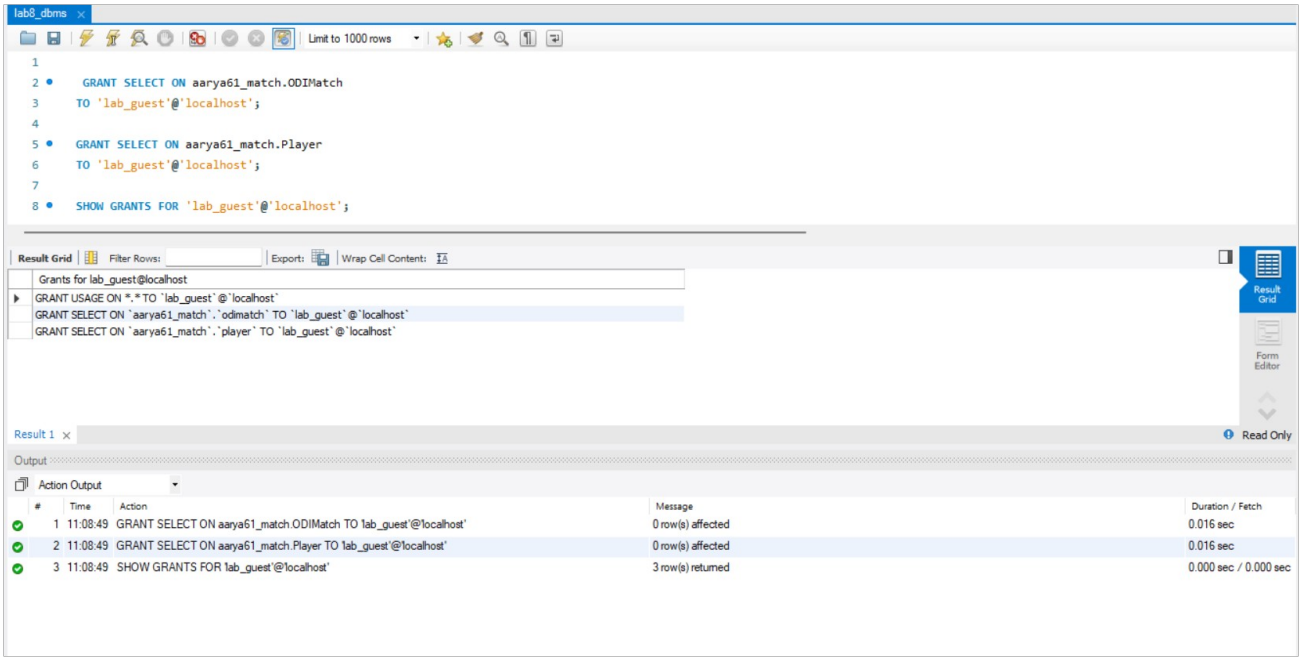


Fig. 1.5: Guest SELECT privileges verified using SHOW GRANTS.

2 [DCL — GRANT with GRANT OPTION]: Grant SELECT, INSERT, and UPDATE privileges on Batting and Bowling tables to 'lab_analyst' WITH GRANT OPTION.

```
-- Root: grant privileges and delegation
GRANT SELECT, INSERT, UPDATE ON aarya61_match.Batting
TO 'lab_analyst'@'localhost' WITH GRANT OPTION;
GRANT SELECT, INSERT, UPDATE ON aarya61_match.Bowling
TO 'lab_analyst'@'localhost' WITH GRANT OPTION;
SHOW GRANTS FOR 'lab_analyst'@'localhost';

-- Lab Analyst: allowed checks, no data changed
SELECT CURRENT_USER();
SELECT * FROM aarya61_match.Batting LIMIT 5;
INSERT INTO aarya61_match.Batting
SELECT * FROM aarya61_match.Batting WHERE 1=0;
UPDATE aarya61_match.Bowling SET MatchID=MatchID WHERE 1=0;

-- Lab Analyst: intentionally DENIED
DELETE FROM aarya61_match.Batting WHERE 1=0;

-- Lab Guest: allowed on a different table due to Q1
SELECT CURRENT_USER();
SELECT * FROM aarya61_match.ODIMatch LIMIT 5;

-- Lab Guest: intentionally DENIED; run statements separately
SELECT * FROM aarya61_match.Batting LIMIT 5;
UPDATE aarya61_match.Bowling SET MatchID=MatchID WHERE 1=0;
INSERT INTO aarya61_match.Batting
(MatchID,PID,POrder,Hout,FOW,NRuns,Mts,Nballs,Fours,Six)
SELECT 0,0,0,'NO','NO',0,0,0,0,0 WHERE 1=0;
```

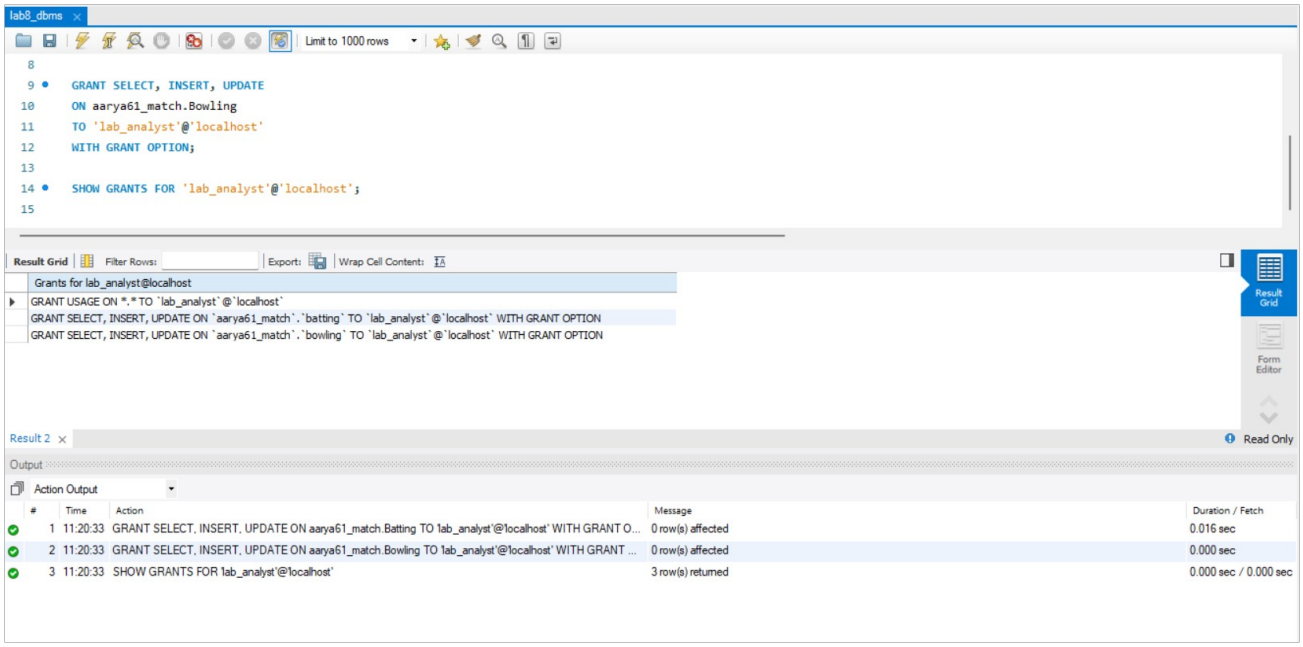


Fig. 2.1: Analyst granted SELECT, INSERT and UPDATE with GRANT OPTION.

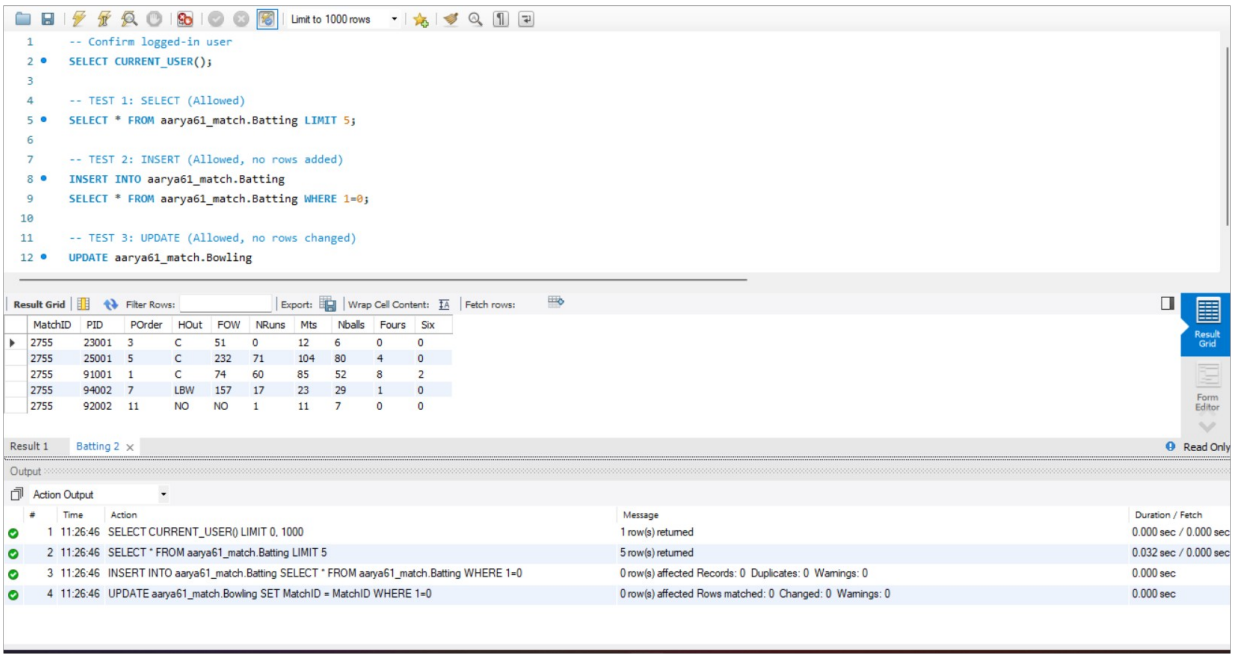


Fig. 2.2: Analyst SELECT, INSERT and UPDATE operations permitted.

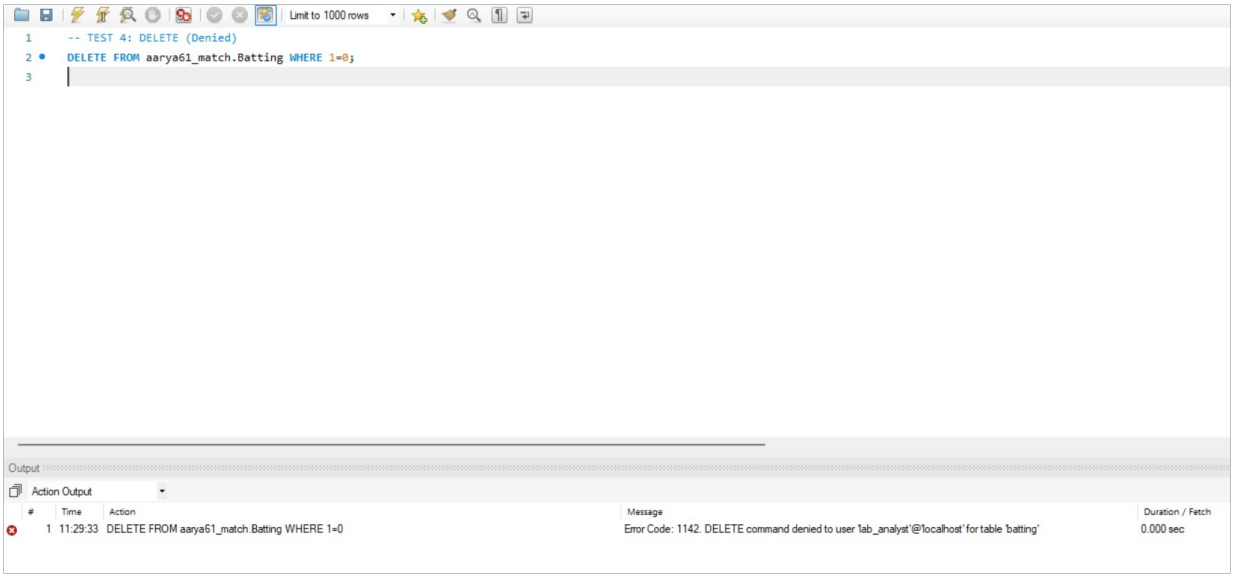


Fig. 2.3: Analyst DELETE operation denied (Error 1142).

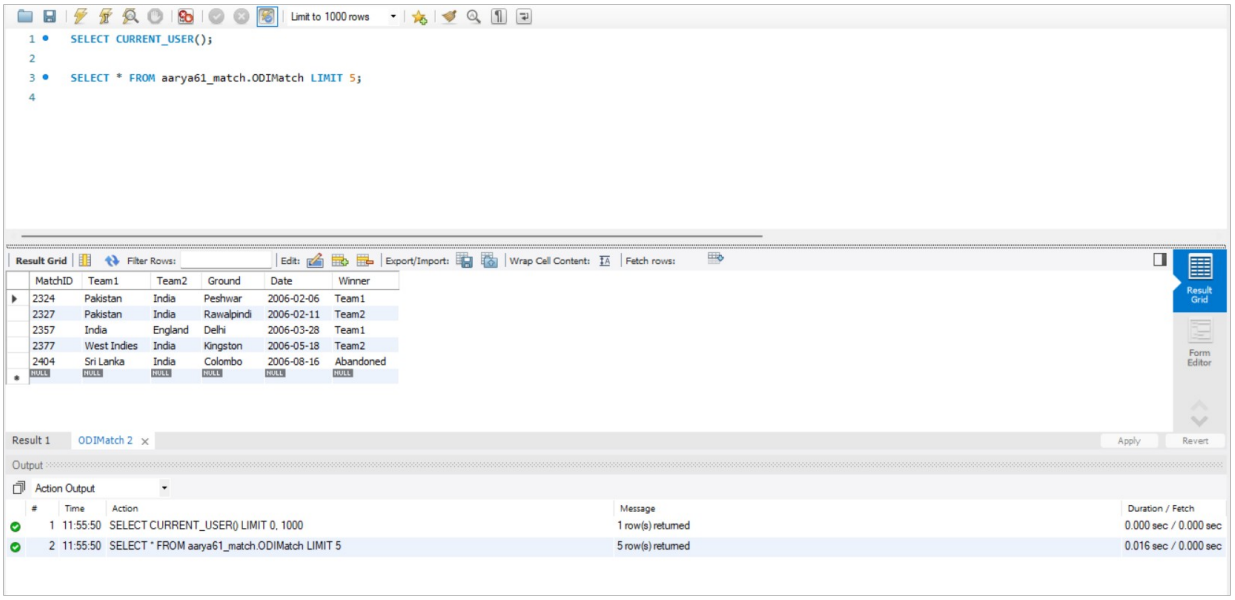


Fig. 2.4: Guest SELECT on ODIMatch permitted.

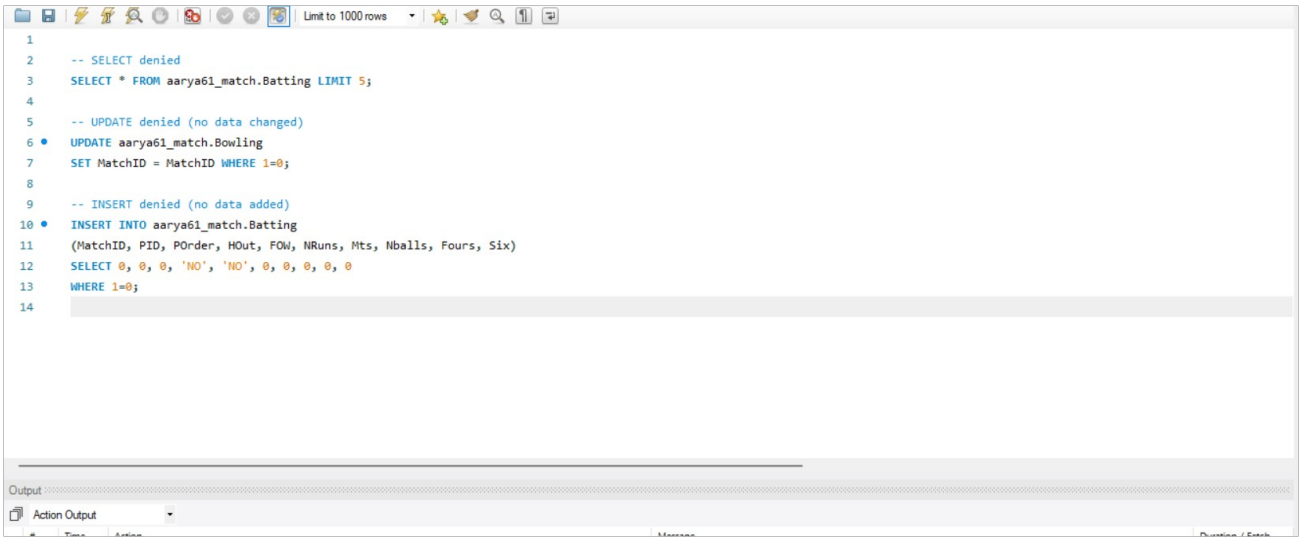


Fig. 2.5: Guest restricted-operation test; Action Output is cropped.

Note: The supplied Guest denied-test screenshot has cropped Action Output logs.

3 [DCL — REVOKE Privileges]: Write SQL statements to revoke UPDATE privileges on Batting from 'lab_analyst' and revoke ALL privileges from 'lab_guest'.

```
-- Root: revoke previously granted privileges
REVOKE UPDATE ON aarya61_match.Batting FROM 'lab_analyst'@'localhost';
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'lab_guest'@'localhost';
SHOW GRANTS FOR 'lab_analyst'@'localhost';
SHOW GRANTS FOR 'lab_guest'@'localhost';

-- Lab Analyst: SELECT and UPDATE Bowling still allowed
SELECT CURRENT_USER();
SELECT * FROM aarya61_match.Batting LIMIT 5;
UPDATE aarya61_match.Bowling SET MatchID=MatchID WHERE 1=0;

-- Lab Analyst: intentionally DENIED
UPDATE aarya61_match.Batting SET NRuns=NRuns WHERE 1=0;

-- Lab Guest: intentionally DENIED
SELECT * FROM aarya61_match.ODIMatch LIMIT 5;
```

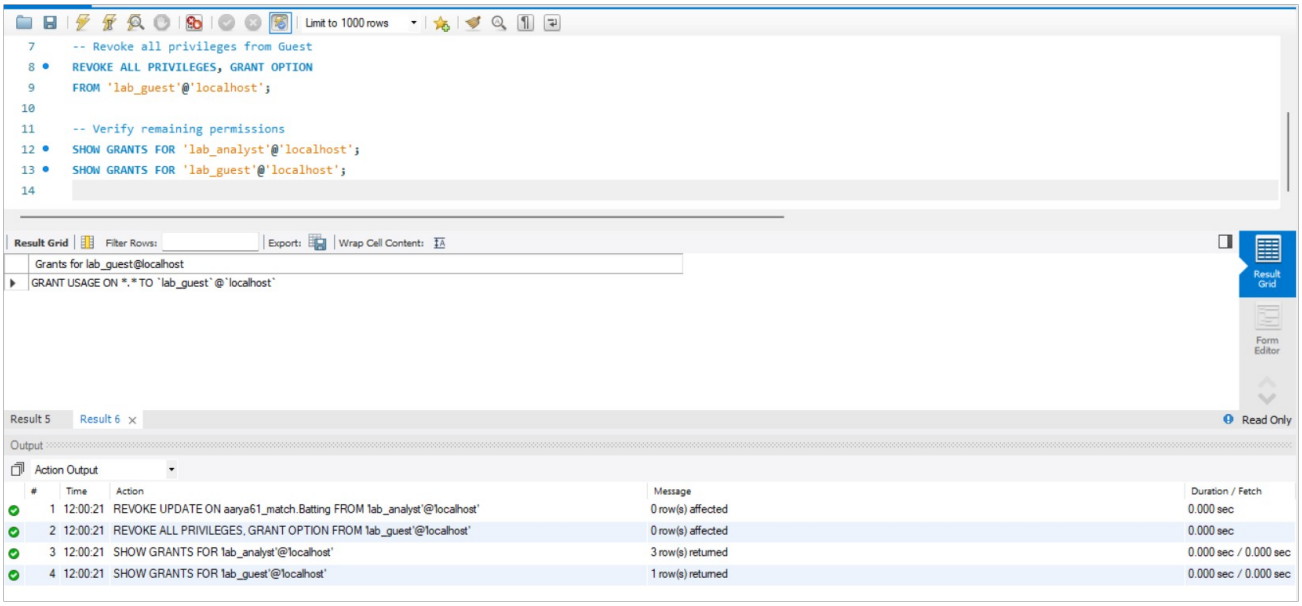


Fig. 3.1: Root revoked Analyst UPDATE and Guest privileges.

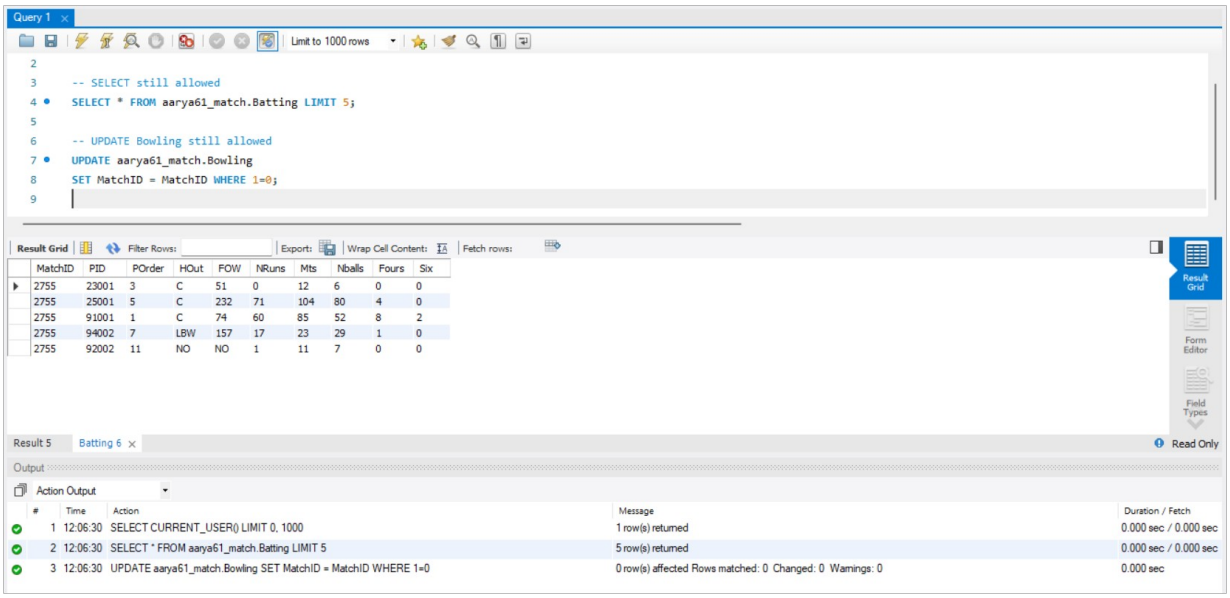


Fig. 3.2: Analyst retains SELECT on Batting and UPDATE on Bowling.

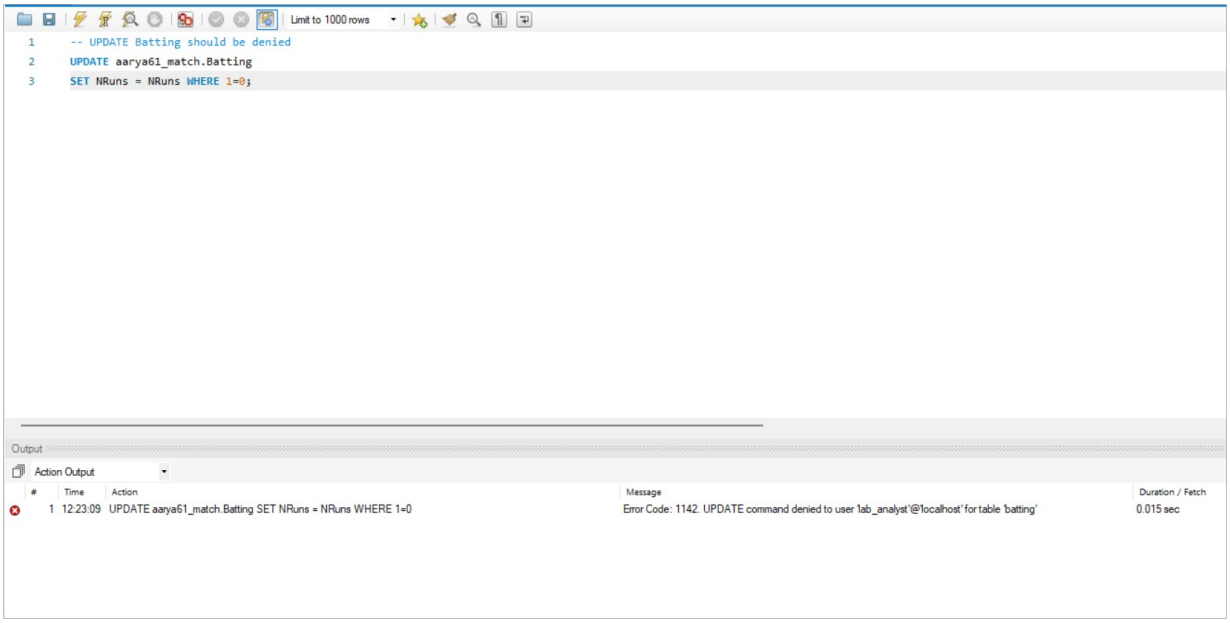


Fig. 3.3: Analyst UPDATE on Batting denied (Error 1142).

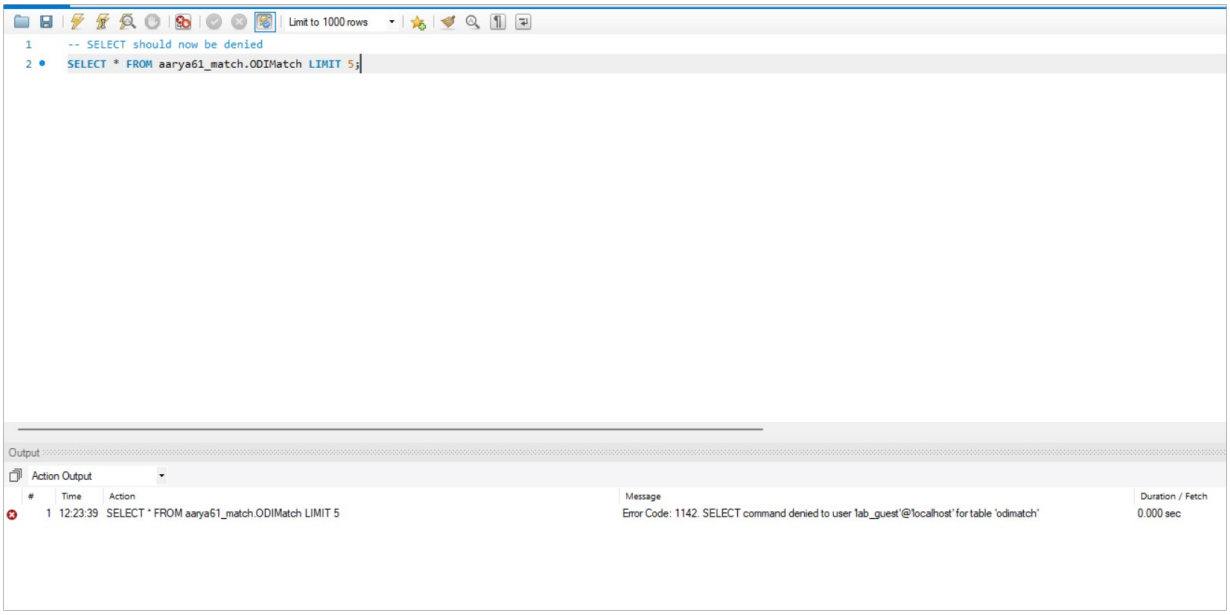


Fig. 3.4: Guest SELECT on ODIMatch denied (Error 1142).

4 [TCL — COMMIT Transaction]: Write a TCL transaction block that inserts a new record into ODIMatch, verifies the record with SELECT, and permanently commits the transaction using COMMIT.

```
-- Root connection
START TRANSACTION;
INSERT INTO aarya61_match.ODIMatch (MatchID,Team1,Team2,Ground,Date,Winner)
VALUES (2800,'India','Australia','Bangalore','2026-10-01','Team1');
SELECT * FROM aarya61_match.ODIMatch WHERE MatchID=2800;
COMMIT;
SELECT * FROM aarya61_match.ODIMatch WHERE MatchID=2800;
```

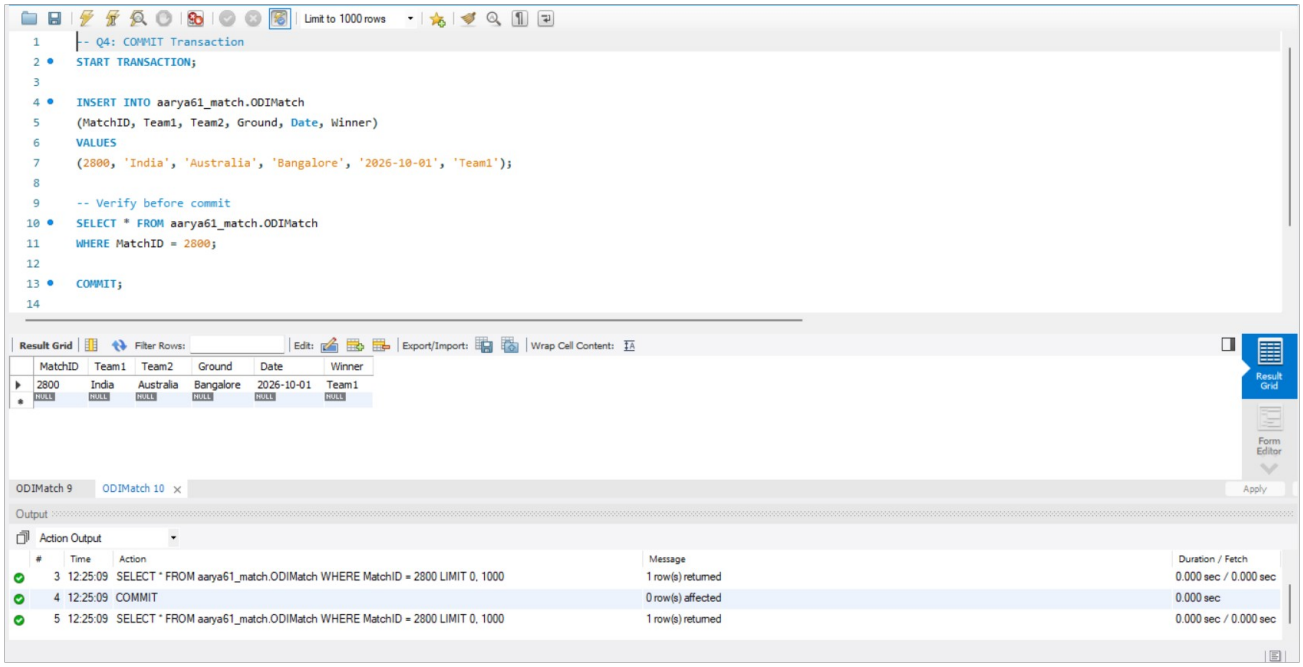


Fig. 4.1: Match 2800 remains saved after COMMIT.

5 [TCL — ROLLBACK Execution]: Write a transaction block that attempts to insert a player into Player and a corresponding batting record into Batting. Simulate an error condition and roll back all changes using ROLLBACK.

```
-- Root connection
START TRANSACTION;
INSERT INTO aarya61_match.Player (PlayerID,LName,FName,Country,Yborn,Bplace)
VALUES (31001,'Sharma','Rohan','India',2001,'Mumbai');
INSERT INTO aarya61_match.Batting (MatchID,PID,POrder,Hout,FOW,NRuns,Mts,Nballs,Fours,Six)
VALUES (2404,31001,8,'NO','NO',999,10,1,0,0);
SELECT * FROM aarya61_match.Batting WHERE PID=31001;
SELECT 'INVALID SCORE - ROLLBACK REQUIRED' AS ValidationStatus;
ROLLBACK;
SELECT * FROM aarya61_match.Player WHERE PlayerID=31001;
SELECT * FROM aarya61_match.Batting WHERE PID=31001;
```

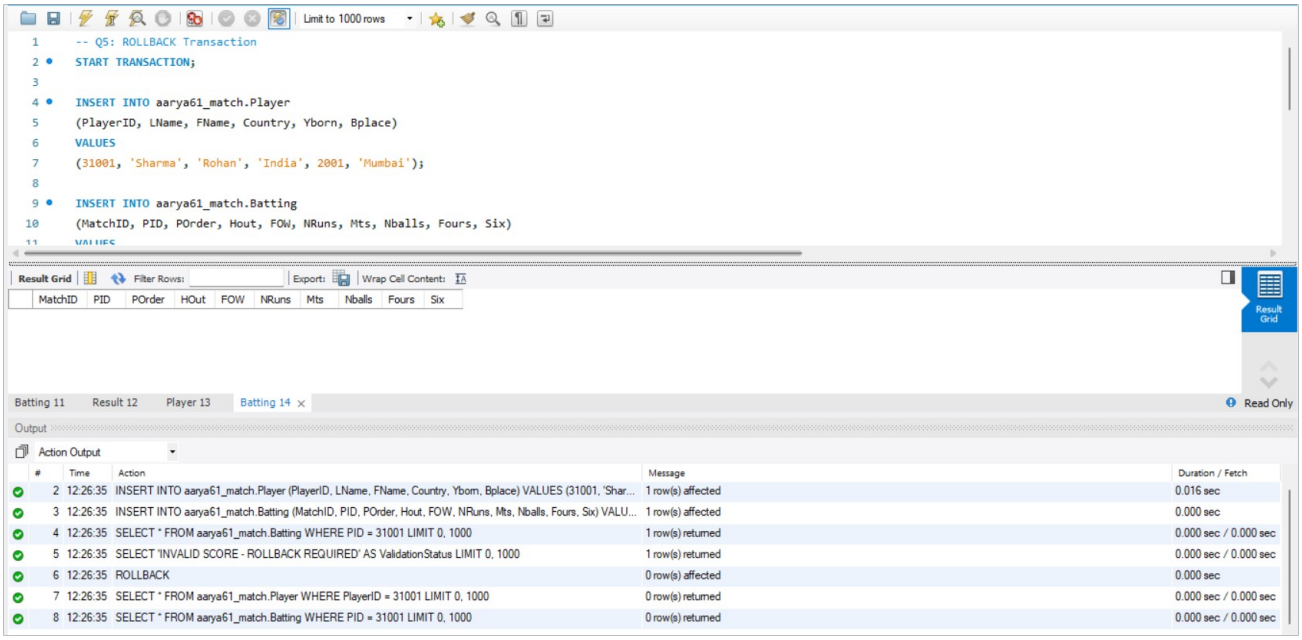


Fig. 5.1: ROLLBACK undoes both Player and Batting insertions.

6 [TCL — SAVEPOINT Usage]: Start a transaction, insert a new match into ODIMatch, set a SAVEPOINT sp_match, insert a record into Batting, and then rollback specifically to SAVEPOINT sp_match.

```
-- Root connection
START TRANSACTION;
INSERT INTO aarya61_match.ODIMatch (MatchID,Team1,Team2,Ground,Date,Winner)
VALUES (2900,'India','England','Chennai','2026-11-01','Team1');
SAVEPOINT sp_match;
INSERT INTO aarya61_match.Batting (MatchID,PID,POrder,Hout,FOW,NRuns,Mts,Nballs,Fours,Six)
SELECT 2900, PlayerID, 1, 'NO', 'NO', 45, 60, 40, 5, 1
FROM aarya61_match.Player ORDER BY PlayerID LIMIT 1;
SELECT * FROM aarya61_match.Batting WHERE MatchID=2900;
ROLLBACK TO SAVEPOINT sp_match;
COMMIT;
SELECT * FROM aarya61_match.ODIMatch WHERE MatchID=2900;
SELECT * FROM aarya61_match.Batting WHERE MatchID=2900;
```

Verification note: Execution evidence was not supplied for this question.

7 [TCL — Multiple Savepoints]: Write a transaction script that inserts 3 new player records, creating savepoints sp1, sp2, and sp3 after each insertion. Roll back to sp2 and commit the final state.

```
-- Root connection
START TRANSACTION;
INSERT INTO aarya61_match.Player (PlayerID,LName,FName,Country,Yborn,Bplace)
VALUES (30001,'Kumar','Bhuvan','India',1990,'Meerut');
SAVEPOINT sp1;
INSERT INTO aarya61_match.Player (PlayerID,LName,FName,Country,Yborn,Bplace)
VALUES (30002,'Singh','Arshdeep','India',1999,'Mohali');
SAVEPOINT sp2;
INSERT INTO aarya61_match.Player (PlayerID,LName,FName,Country,Yborn,Bplace)
VALUES (30003,'Yadav','Kuldeep','India',1994,'Kanpur');
SAVEPOINT sp3;
SELECT PlayerID,FName,LName FROM aarya61_match.Player WHERE PlayerID IN (30001,30002,30003);
ROLLBACK TO SAVEPOINT sp2;
COMMIT;
SELECT PlayerID,FName,LName FROM aarya61_match.Player WHERE PlayerID IN (30001,30002,30003);
```

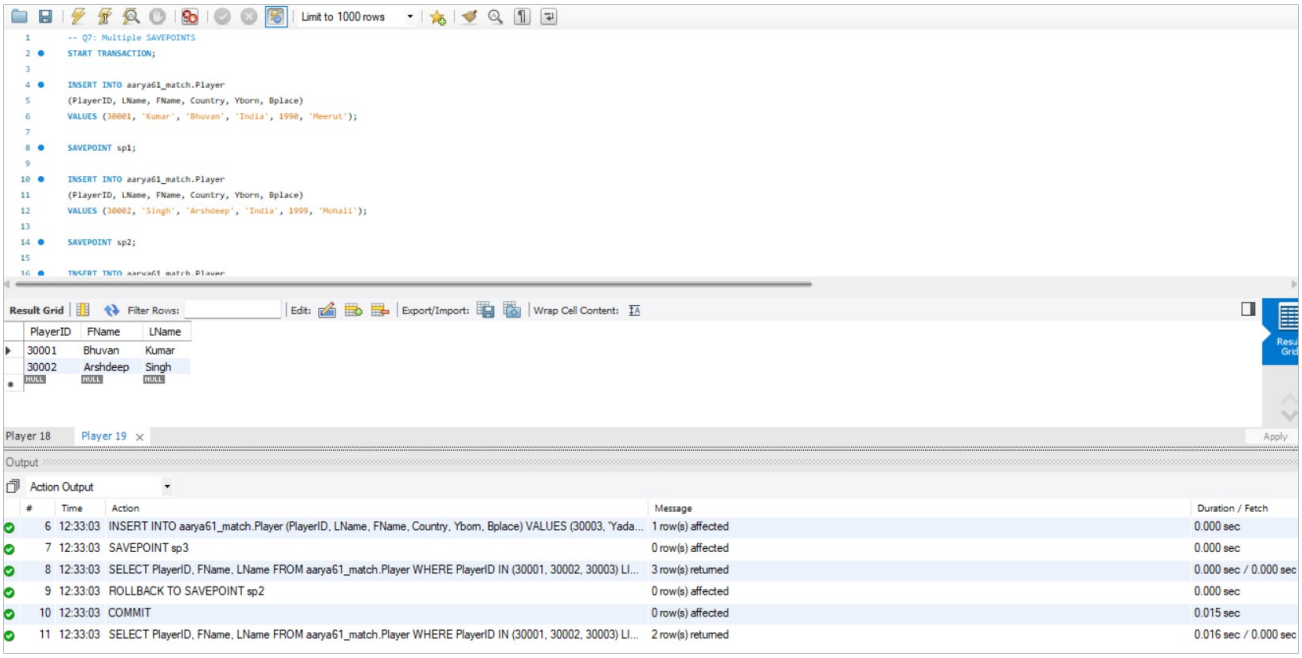


Fig. 7.1: ROLLBACK TO sp2 retains only the first two players.

8 [TCL — Isolation Levels]: Write TCL statements to set the transaction isolation level to READ COMMITTED before updating player records in a multi-user lab environment.

```
-- Root connection
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT @@transaction_isolation;
START TRANSACTION;
UPDATE aarya61_match.Player SET Country='India' WHERE PlayerID=27001;
COMMIT;
SELECT PlayerID,FName,Country FROM aarya61_match.Player WHERE PlayerID=27001;
```

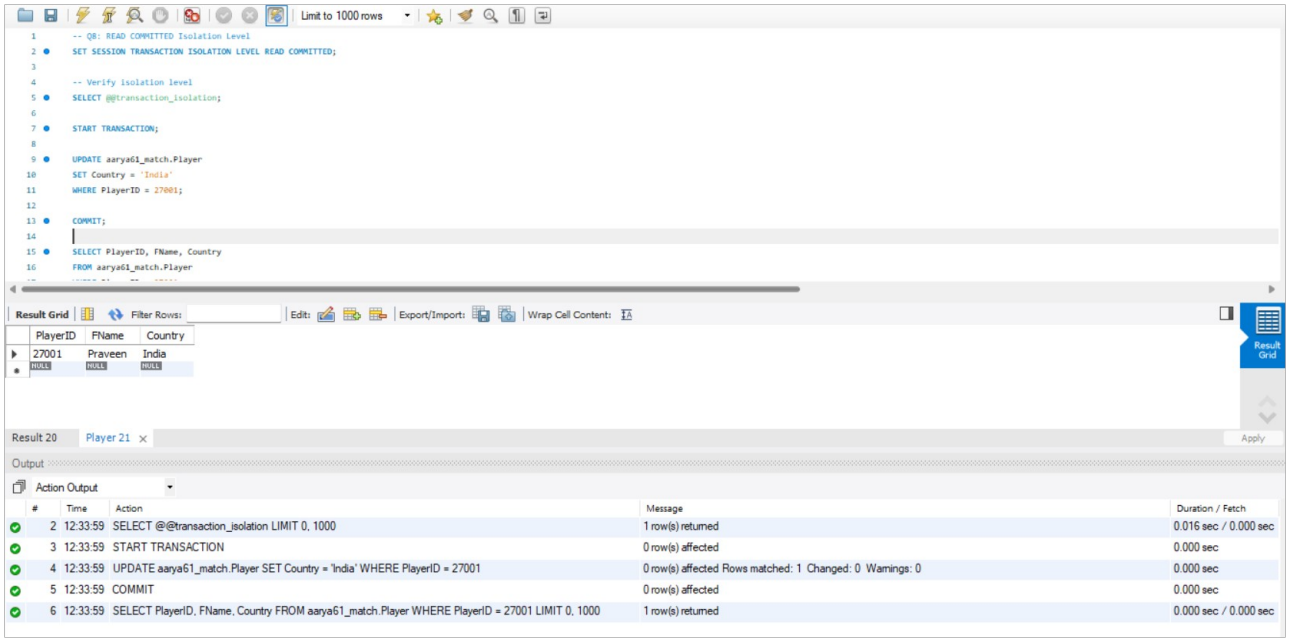


Fig. 8.1: READ COMMITTED set; player transaction committed.

9 [TCL — Partial Update & Commit]: Update the Winner attribute of match 2404 to 'India', set a savepoint sp_update, modify FTest year for player 27001, and execute COMMIT.

```
-- Root connection: schema check performed separately
SHOW COLUMNS FROM aarya61_match.Player LIKE 'FTest';
-- FTest exists (int)
START TRANSACTION;
UPDATE aarya61_match.ODIMatch SET Winner='India' WHERE MatchID=2404;
SAVEPOINT sp_update;
UPDATE aarya61_match.Player SET FTest=2013 WHERE PlayerID=27001;
COMMIT;
SELECT MatchID,Winner FROM aarya61_match.ODIMatch WHERE MatchID=2404;
SELECT PlayerID,FName,FTest FROM aarya61_match.Player WHERE PlayerID=27001;
```

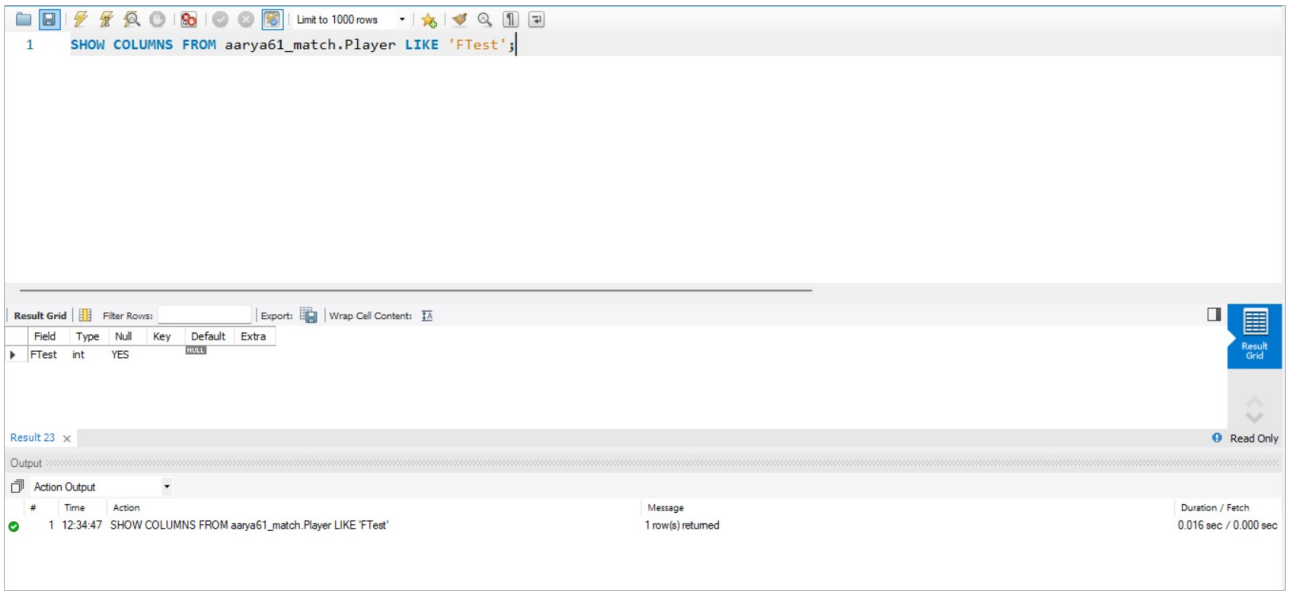


Fig. 9.1: The FTest column exists in the Player table.

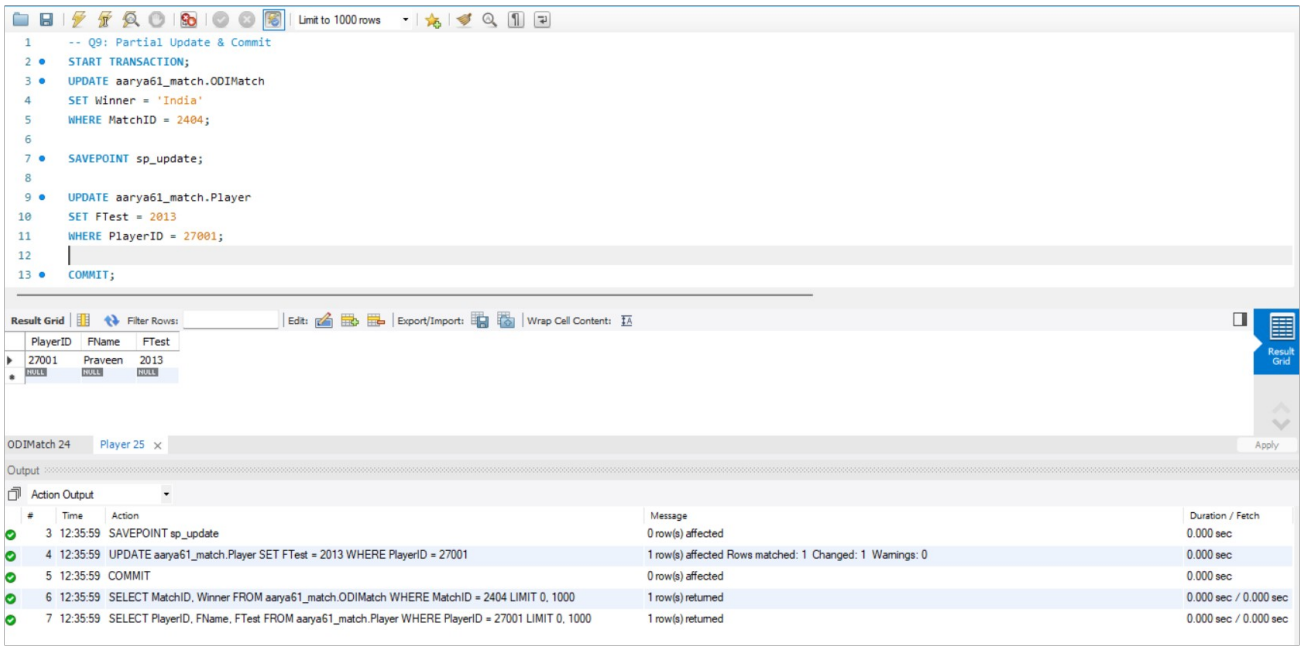


Fig. 9.2: Match Winner and player FTest changes committed.

10 [DCL — Restricted View Grant]: Create a restricted view Indian_Players_View and grant SELECT access on this view to user 'cricket_fan', keeping the underlying Player table hidden.

```
-- Root connection
CREATE OR REPLACE VIEW aarya61_match.Indian_Players_View AS
SELECT PlayerID,FName,LName,Country,Yborn
FROM aarya61_match.Player WHERE Country='India';
CREATE USER IF NOT EXISTS 'cricket_fan'@'localhost' IDENTIFIED BY 'FanPass123!';
GRANT SELECT ON aarya61_match.Indian_Players_View TO 'cricket_fan'@'localhost';
SHOW GRANTS FOR 'cricket_fan'@'localhost';

-- Cricket Fan connection: allowed
SELECT CURRENT_USER();
SELECT * FROM aarya61_match.Indian_Players_View LIMIT 5;

-- Cricket Fan connection: intentionally denied
SELECT * FROM aarya61_match.Player LIMIT 5;
```

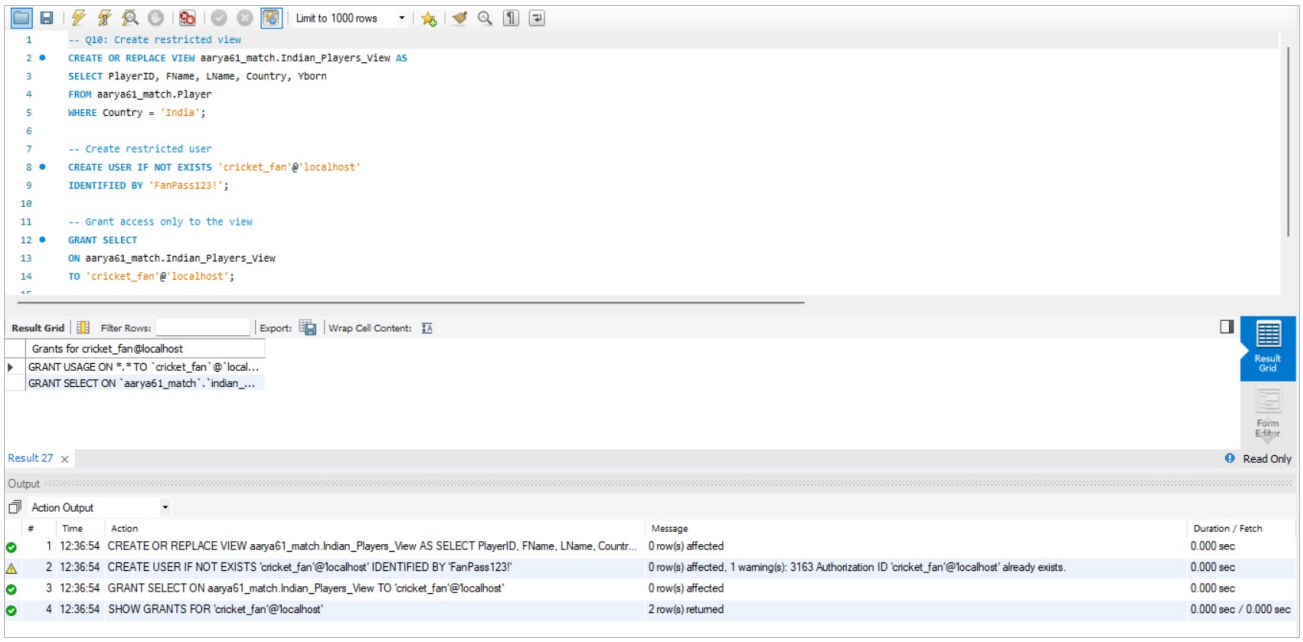


Fig. 10.1: Restricted view created and SELECT granted.

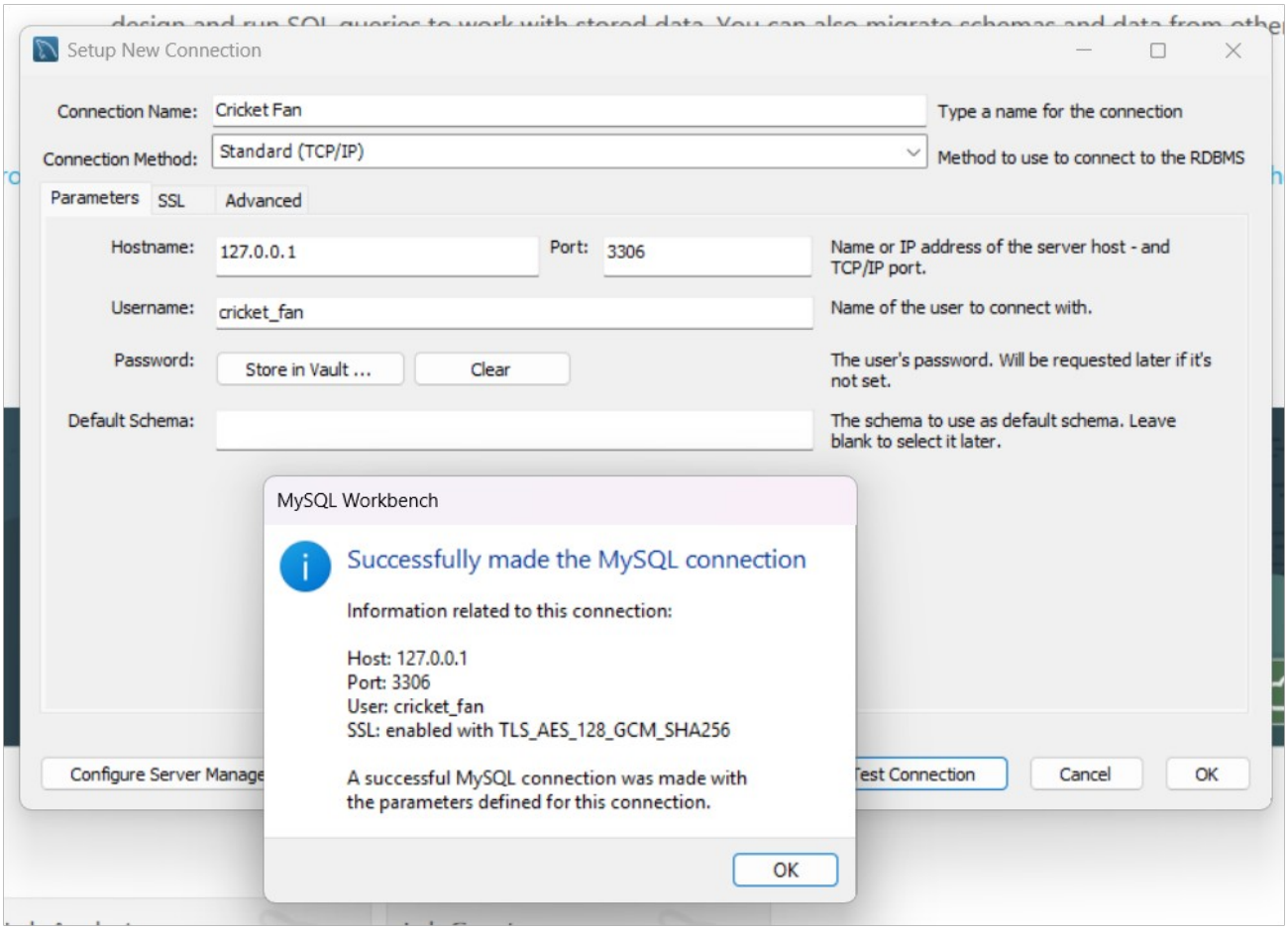


Fig. 10.2: Cricket Fan connection tested successfully.

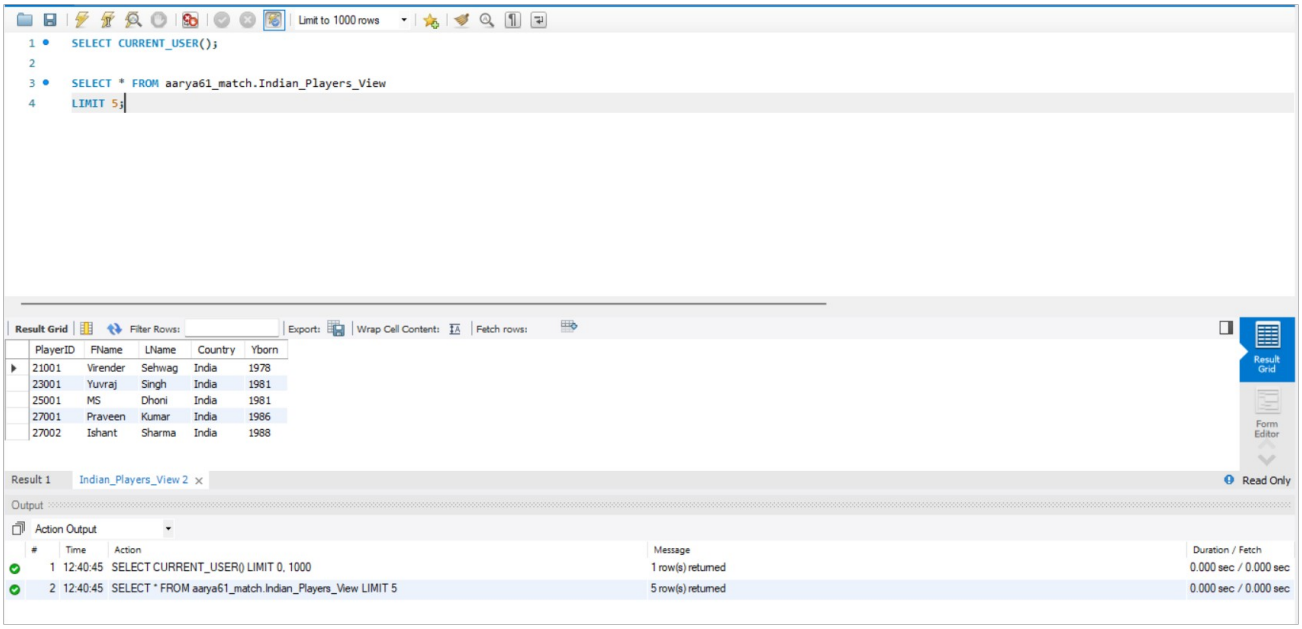


Fig. 10.3: Cricket Fan can read Indian_Players_View.

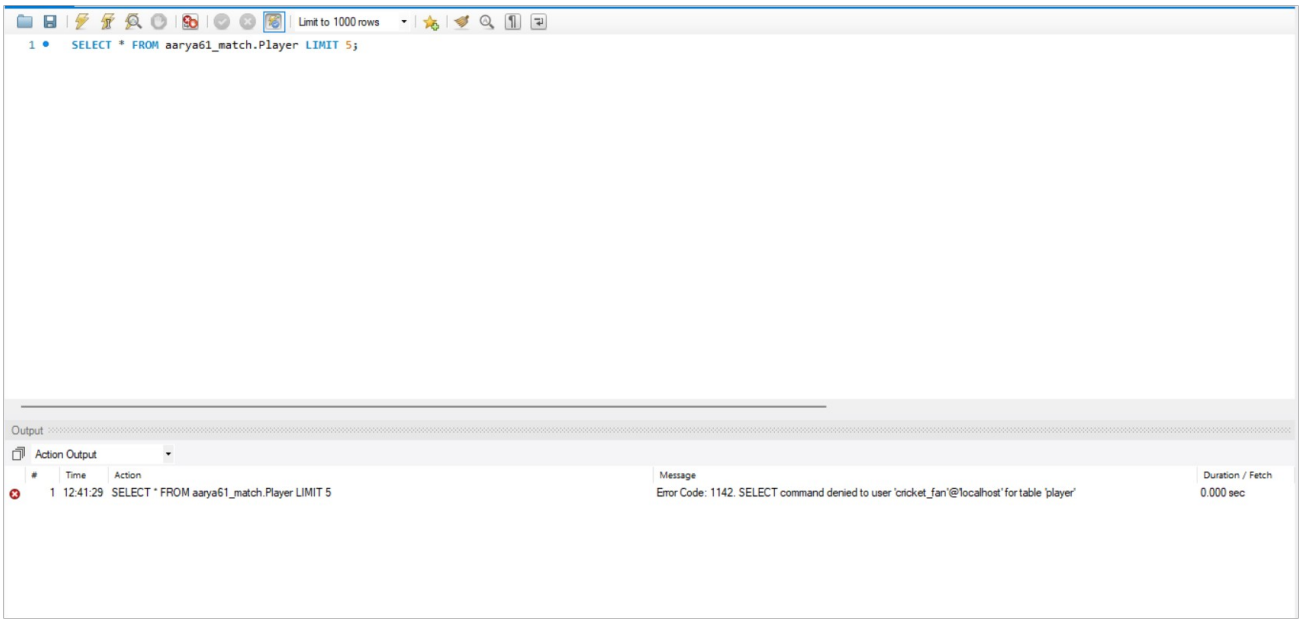


Fig. 10.4: Direct Player table access denied (Error 1142).

11 [DCL — Audit Privileges]: Write administrative SQL commands (SHOW GRANTS) to inspect and verify all active privileges granted to 'lab_analyst'@'localhost'.

```
-- Root connection
SHOW GRANTS FOR 'lab_analyst'@'localhost';
```

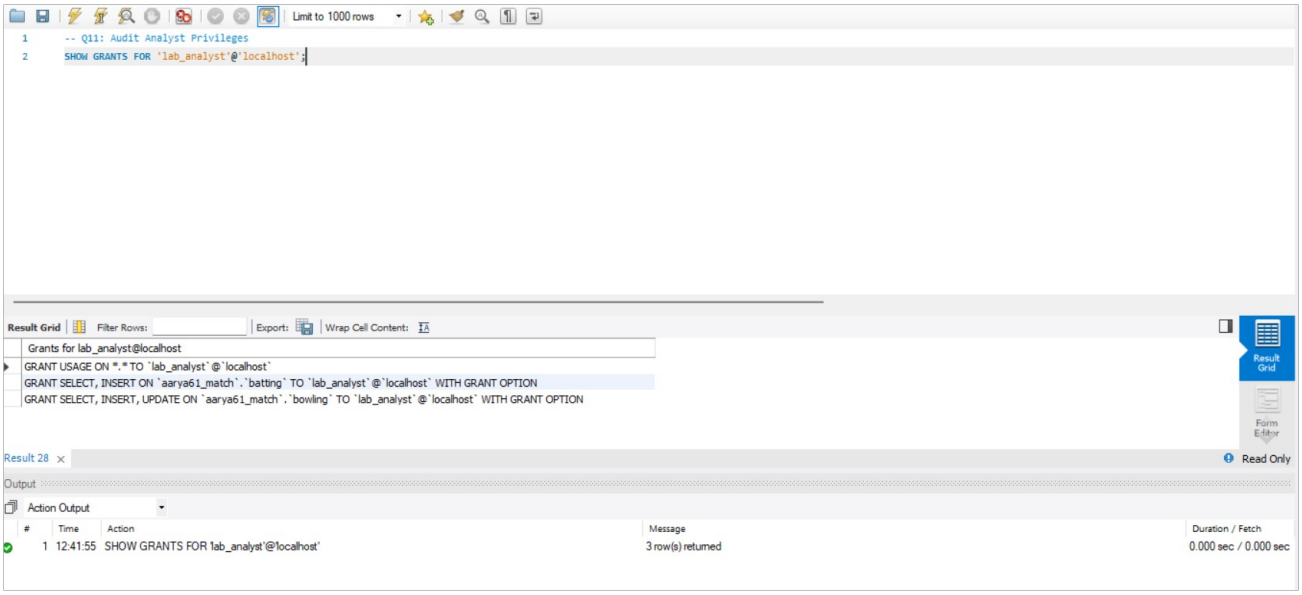


Fig. 11.1: Analyst privileges audited using SHOW GRANTS.

12 [TCL — Deletion with Rollback]: Begin a transaction, delete bowling records for MatchID = 2689, set savepoint sp_del1, delete batting records for MatchID = 2689, rollback to sp_del1, and commit.

```
-- Root connection. Note: Bowling deletion is committed, so this changes data.
SELECT * FROM aarya61_match.Bowling WHERE MatchID=2689;
SELECT * FROM aarya61_match.Batting WHERE MatchID=2689;
START TRANSACTION;
DELETE FROM aarya61_match.Bowling WHERE MatchID=2689;
SAVEPOINT sp_del1;
DELETE FROM aarya61_match.Batting WHERE MatchID=2689;
ROLLBACK TO SAVEPOINT sp_del1;
COMMIT;
SELECT * FROM aarya61_match.Bowling WHERE MatchID=2689;
SELECT * FROM aarya61_match.Batting WHERE MatchID=2689;
```

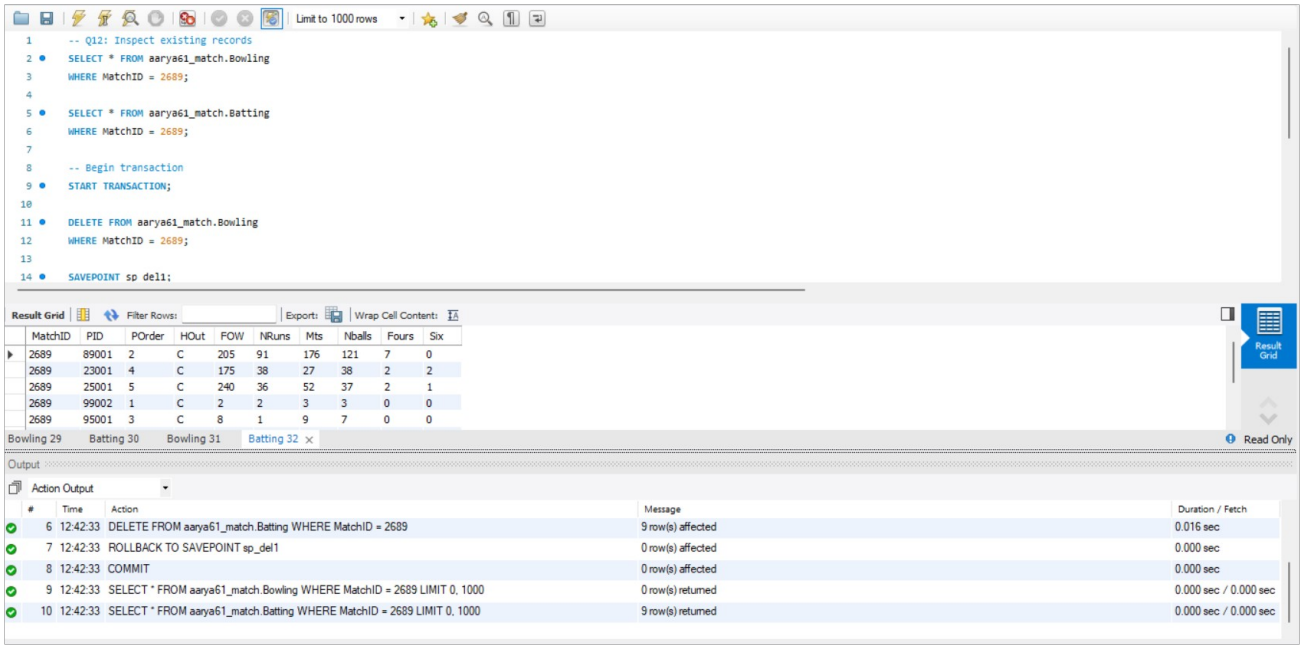


Fig. 12.1: Bowling deletion committed; Batting rows retained.

13 [DCL & TCL — Role Scenario]: Create a role 'data_entry_role', grant INSERT and UPDATE on Batting to this role, assign the role to a user, and execute a committed insertion under that role session.

```
-- Root connection
CREATE ROLE IF NOT EXISTS 'data_entry_role';
GRANT INSERT, UPDATE ON aarya61_match.Batting TO 'data_entry_role';
GRANT 'data_entry_role' TO 'lab_analyst'@'localhost';
SET DEFAULT ROLE 'data_entry_role' TO 'lab_analyst'@'localhost';
SHOW GRANTS FOR 'data_entry_role';

-- Lab Analyst connection: role and committed transaction
SET ROLE 'data_entry_role';
SELECT CURRENT_ROLE();
START TRANSACTION;
INSERT INTO aarya61_match.Batting (MatchID,PID,POrder,Hout,FOW,NRuns,Mts,Nballs,Fours,Six)
VALUES (2800,30002,3,'NO','NO',45,60,40,5,1);
UPDATE aarya61_match.Batting SET NRuns=NRuns WHERE MatchID=2800 AND PID=30002;
COMMIT;
SELECT * FROM aarya61_match.Batting WHERE MatchID=2800 AND PID=30002;
```

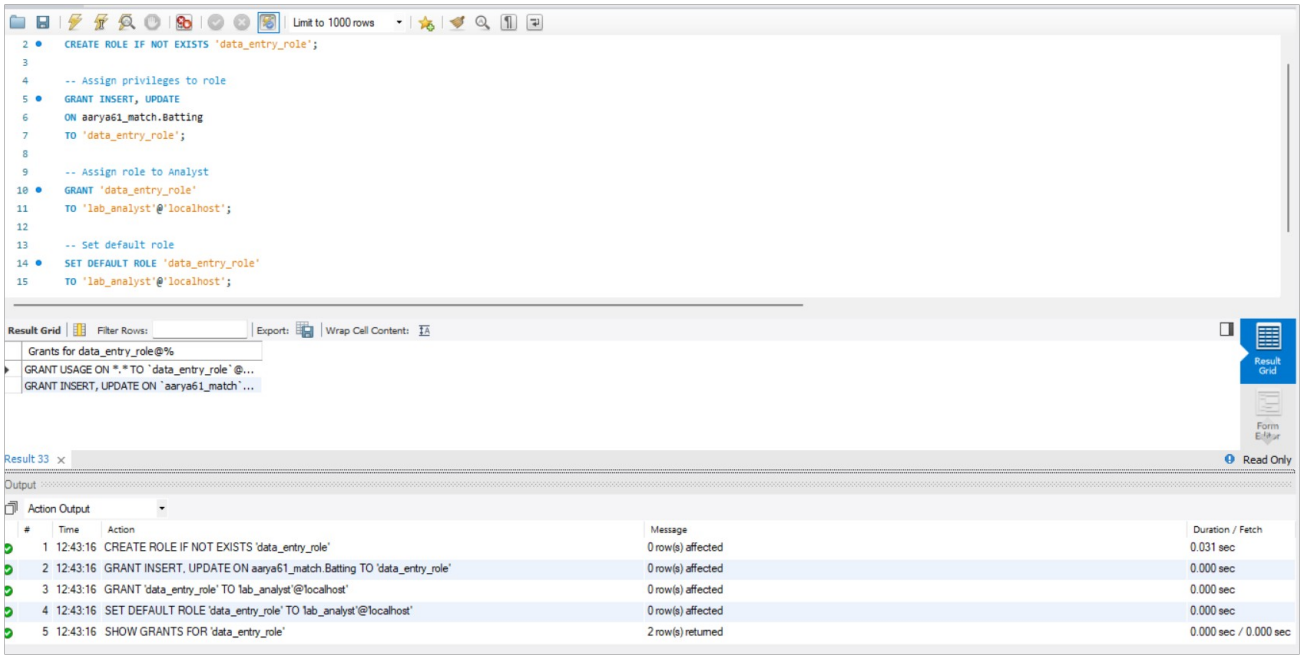


Fig. 13.1: Data entry role created, granted and assigned.

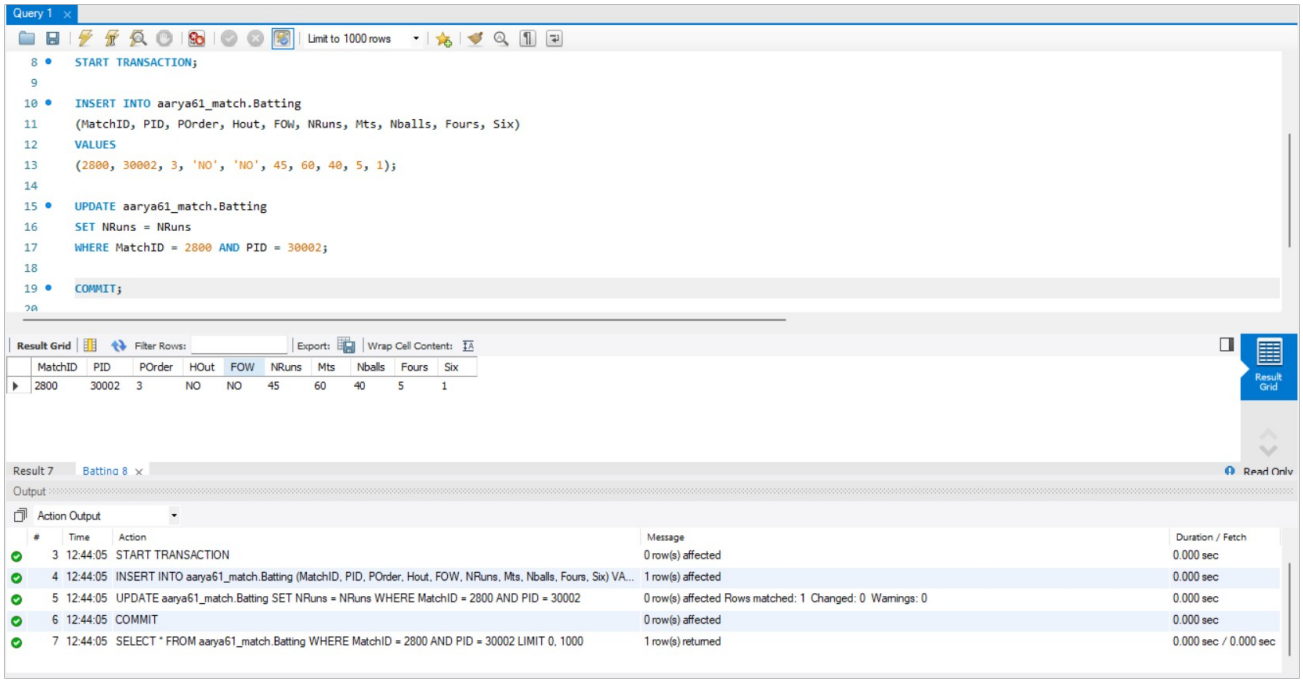


Fig. 13.2: Role-based Batting INSERT successfully committed.